# Hardware Acceleration of the Number Theoretic Transform (NTT) for Post-Quantum Cryptography

**Prepared By:** Maturi Mrudul
**Roll No:** 250002038
Department of Electrical Engineering, Indian Institute of Technology Indore

October 2026

**Abstract**

The imminent arrival of large-scale quantum computers threatens the security of classical public-key cryptography. To mitigate this, the National Institute of Standards and Technology (NIST) has standardized lattice-based cryptographic algorithms, such as ML-KEM (Kyber) and ML-DSA (Dilithium). The most computationally intensive operation in these algorithms is polynomial multiplication in a cyclotomic ring. This report details the methodology and architecture for accelerating this operation using a hardware-based Number Theoretic Transform (NTT) on a Field Programmable Gate Array (FPGA). By leveraging hardware-specific optimizations like Montgomery Reduction and conflict-free memory banking, the proposed architecture significantly reduces computational latency compared to software implementations.

## 1 Problem Statement

Lattice-based cryptographic schemes derive their security from the hardness of the Learning With Errors (LWE) and Module-LWE problems. The mathematical bottleneck in these schemes lies in polynomial arithmetic, specifically multiplying two high-degree polynomials within the ring:

$$\mathcal{R}_q = \mathbb{Z}_q[x]/(x^n + 1)$$

where $n = 256$ and $q$ is a prime modulus (e.g., $q = 3329$ for Kyber).

Executing polynomial multiplication via the standard "schoolbook" method requires $\mathcal{O}(n^2)$ operations. For $n = 256$, this translates to 65,536 modular multiplications per pair of polynomials. In resource-constrained embedded systems or high-throughput servers, executing this natively in software results in severe latency and excessive power consumption, preventing the system from meeting real-time cryptographic demands.

## 2 Mathematical Foundation: What the Project Does

To resolve the computational bottleneck, this project implements the Number Theoretic Transform (NTT). The NTT is a finite-field equivalent of the Fast Fourier Transform (FFT) that reduces the multiplication complexity from $\mathcal{O}(n^2)$ to $\mathcal{O}(n \log n)$.

### 2.1 Negacyclic Convolution and the CRT

Standard polynomial multiplication over a finite field computes a cyclic convolution, corresponding to modulo $(x^n - 1)$. However, lattice cryptography strictly requires negacyclic convolution, corresponding to modulo $(x^n + 1)$.

To achieve this mathematically, the algorithm employs the Chinese Remainder Theorem (CRT). We define a primitive $2n$-th root of unity, $\psi$, such that $\psi^n \equiv -1 \pmod{q}$. This property allows us to recursively factor the modulus $x^n + 1$ as a difference of squares:

$$x^n + 1 = (x^{n/2} - \psi^{n/2})(x^{n/2} + \psi^{n/2})$$

The NTT algorithm hardware evaluates the polynomial at these roots layer by layer. Once transformed into the NTT domain, the two polynomials are multiplied pointwise in $\mathcal{O}(n)$ time, and an Inverse NTT (INTT) is applied to recover the final multiplied polynomial.

# 3 Hardware Architecture: How We Implement It

The project physically maps the NTT mathematical equations into digital logic gates on an FPGA using Verilog HDL. The architecture is divided into three major modules.

## 3.1 Pipelined Butterfly Unit (BU)

The atomic computational element of the NTT is the butterfly operation. The hardware processes two coefficients simultaneously per clock cycle using a unified, reconfigurable datapath:

- **Forward NTT:** The datapath executes the Cooley-Tukey (CT) algorithm: $a_{out} = a + \psi \cdot b \pmod{q}$ and $b_{out} = a - \psi \cdot b \pmod{q}$.
- **Inverse NTT:** The datapath switches to the Gentleman-Sande (GS) algorithm: $a_{out} = a + b \pmod{q}$ and $b_{out} = (a - b) \cdot \psi \pmod{q}$.

Registers are inserted at intermediate stages (e.g., after multiplication and before addition) to shorten the critical path and achieve a high maximum clock frequency ($F_{max}$).

## 3.2 Montgomery Reduction Datapath

In hardware, true division by an arbitrary prime $q$ is extremely resource-intensive. To bypass this, we use Montgomery Reduction. By scaling values by a factor of $R = 2^{16}$, division by $q$ is mathematically replaced by division by $2^{16}$.

In Verilog, division by $2^{16}$ requires zero logic gates—it is achieved by simply routing the top 16 bits of a 32-bit wire. The reduction pipeline is implemented as follows:

1. Extract the lower 16 bits of the multiplier's output product.
2. Multiply this lower half by a precomputed constant $Q_{INV} = -q^{-1} \pmod{2^{16}}$.
3. Multiply the truncated result by the prime $q$.
4. Subtract this value from the original product, and route the upper 16 bits to the output.

This reduces a complex modulo operation into exactly two DSP multiplications and one subtraction.

### 3.3 Conflict-Free Memory Banking

The NTT algorithm requires reading and writing pairs of coefficients. If both coefficients reside in the same physical Block RAM (BRAM), a memory collision occurs, stalling the pipeline. To resolve this, coefficients are distributed across two independent BRAM banks based on the parity (XOR sum) of their address bits. This banking strategy guarantees that the required coefficient pairs always reside in separate banks, enabling the Butterfly Unit to process one operation per clock cycle continuously.

## 4 Detailed Implementation Procedure

The execution and validation of the project follow a structured, multi-phase hardware design methodology.

### 4.1 Phase 1: Golden Model Development

A bit-accurate software model is developed in Python. This model mimics the exact bit-widths, Montgomery reduction steps, and twiddle factor addresses used in the hardware. The model's outputs are strictly verified against the official NIST FIPS 203 (ML-KEM) and FIPS 204 (ML-DSA) test vectors to ensure mathematical correctness.

### 4.2 Phase 2: Register Transfer Level (RTL) Design

The system is written in Verilog HDL. The design encompasses:

- **Datapath:** The DSP-based Montgomery multipliers and add/sub units.
- **Memory Controllers:** Dual-port BRAM instantiation for coefficient storage, and LUT-based ROM for storing the precomputed twiddle factors ($\psi$ powers).
- **Finite State Machine (FSM):** A central controller that orchestrates the layers of the NTT. It updates stride lengths, generates RAM read/write addresses, and controls pipeline stalls or flushes.

### 4.3 Phase 3: RTL Simulation and Verification

Using an industry-standard simulator (e.g., ModelSim or Questa), Verilog testbenches are constructed. Randomized polynomials are fed into the RTL. The output waveforms and resulting memory states are captured and compared automatically against the Python golden model's output. Edge cases, such as coefficients equaling $0$ or $q - 1$, are heavily tested.

### 4.4 Phase 4: Synthesis and Physical Implementation

The verified RTL is synthesized using FPGA vendor tools (e.g., Xilinx Vivado or Intel Quartus). During this phase:

- **Resource Mapping:** The tool is constrained to map the Verilog multipliers directly into dedicated DSP slices rather than using general-purpose LUTs.
- **Timing Analysis:** Static Timing Analysis (STA) is performed to ensure the pipelined datapath meets the target clock frequency without setup or hold time violations.

### 4.5 Phase 5: On-Board System Integration

In the final phase, the NTT accelerator IP is instantiated alongside a soft-core processor (such as MicroBlaze or Nios II) on the FPGA fabric via an AXI-4 interface. The execution flow is as follows:

1. The soft-core processor writes the initial polynomial coefficients into the accelerator's BRAM.
2. The processor asserts a `start` signal over the AXI bus.
3. The hardware FSM executes the $O(n \log n)$ butterflies completely independently of the processor.
4. Upon completing the 7 or 8 layers (depending on the specific algorithm), the accelerator raises a `done` interrupt.
5. The processor reads the transformed coefficients back from the BRAM for further cryptographic processing (e.g., pointwise multiplication or sampling).

## 5 Conclusion

This project successfully accelerates the most demanding component of post-quantum lattice-based cryptography. By utilizing a pipelined butterfly architecture, Montgomery modular reduction, and a conflict-free memory schema, the FPGA implementation drastically reduces the cycle count for polynomial multiplication. This hardware accelerator serves as a critical IP block for integrating secure, quantum-resistant cryptography into high-performance computing environments.